

Distributed Systems

Lecture17: Clock Synchronization

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Physical Clocks

- Sometimes we simply need the exact time, not just an ordering.
- Universal Coordinated Time (UTC):
  - Based on the number of transitions per second of the cesium 133 atom (pretty accurate).
  - At present, the real time is taken as the average of some 50 cesium clocks around the world.
  - Introduces a leap second from time to time to compensate that days are getting longer.

Note 17.1.

UTC is broadcast through short-wave radio and satellite. Satellites can give an accuracy of about ±0.5 ms.

Clock Synchronization

- **Precision**
  - The goal is to keep the deviation between two clocks on any two machines within a specified bound, known as the precision π:

$$\forall t, \forall p, q : |C_p(t) - C_q(t)| \leq \pi$$

  - with $C_p(t)$ the computed clock time of machine p at UTC time t
- **Accuracy**
  - In the case of accuracy, we aim to keep the clock bound to a value α:

$$\forall t, \forall p : |C_p(t) - t| \leq \alpha$$

- **Synchronization:**
  - Internal synchronization: keep clocks precise
  - External synchronization: keep clocks accurate

Clock Drift

- **Clock Specifications:**
  - A clock comes specified with its maximum clock drift rate ρ.
  - F (t) denotes oscillator frequency of the hardware clock at time t
  - F is the clock's ideal (constant) frequency ⇒ living up to specifications:

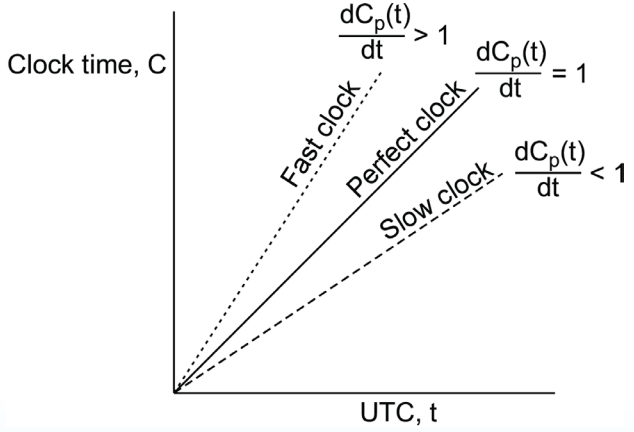
$$\forall t : (1 - \rho) \leq \frac{F(t)}{F} \leq (1 + \rho)$$

- **Observation**
- By using hardware interrupts we couple a software clock to the hardware clock, and thus also its clock drift rate:

$$C_p(t) = \frac{1}{F} \int_0^t F(t)dt \Rightarrow \frac{dC_p(t)}{dt} = \frac{F(t)}{F}$$

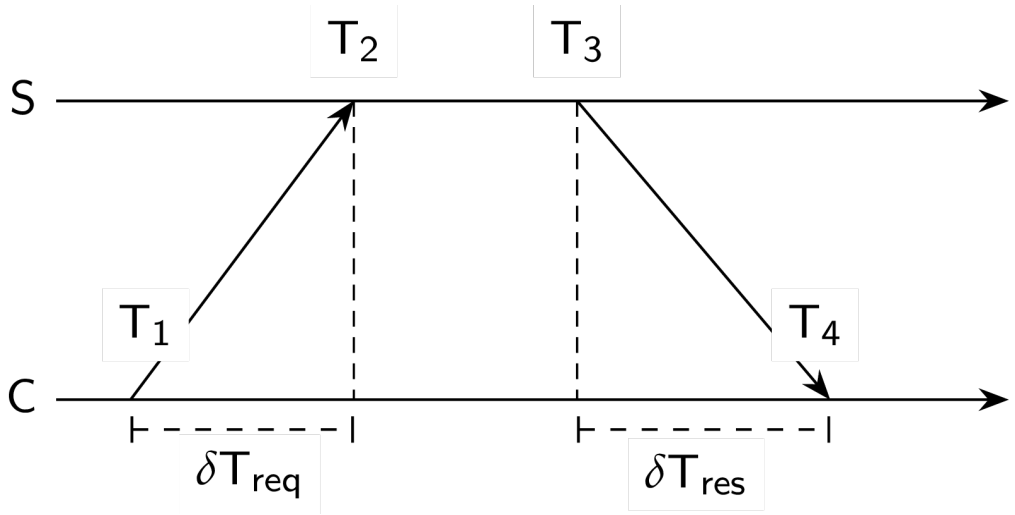
$$\Rightarrow \forall t : 1 - \rho \leq \frac{dC_p(t)}{dt} \leq 1 + \rho$$

Fast, perfect, slow clocks



Detecting And Adjusting Incorrect Times

- Getting the current time from a timeserver



- Computing the relative offset θ and delay δ

Assumption: $\delta T_{req} = T_2 - T_1 \approx T_4 - T_3 = \delta T_{res}$

$$\theta = T_3 + ((T_2 - T_1) + (T_4 - T_3))/2 - T_4 = ((T_2 - T_1) + (T_3 - T_4))/2$$

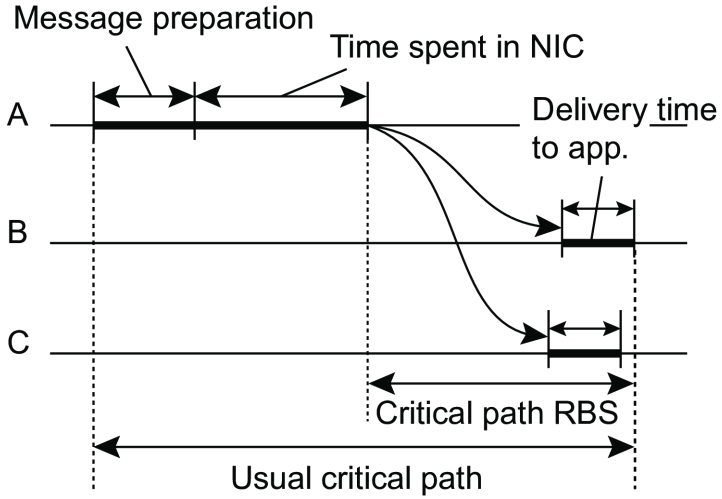
$$\delta = ((T_4 - T_1) - (T_3 - T_2))/2$$

Reference Broadcast Synchronization

- A node broadcasts a reference message m ⇒ each receiving node p
- records the time Tp,m that it received m.
- Note: Tp,m is read from p's local clock.
- **Problem: averaging will not capture**
- RBS minimizes critical path drift ⇒ use linear regression

NO: $Offset[p, q](t) = \frac{\sum_{k=1}^{M}(T_{p,k} - T_{q,k})}{M}$

YES: $Offset[p, q](t) = \alpha t + \beta$



The Happened-before Relationship

- What usually matters is not that all processes agree on exactly what time it is, but that they agree on the order in which events occur. Requires a notion of ordering.
- **The happened-before relation**
  - If a and b are two events in the same process, and a comes before b, then $a \rightarrow b$.
  - If a is the sending of a message, and b is the receipt of that message, then $a \rightarrow b$
  - If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$
- This introduces a partial ordering of events in a system with concurrently operating processes.

Logical Clocks

- How do we maintain a global view of the system's behavior that is consistent with the happened-before relation?
- Attach a timestamp C(e) to each event e, satisfying the following properties:

  P1) If a and b are two events in the same process, and $a \rightarrow b$, then we demand that $C(a) < C(b)$.

  P2) If a corresponds to sending a message m, and b to the receipt of that message, then also $C(a) < C(b)$.

- How to attach a timestamp to an event when there's no global clock $\Rightarrow$ maintain a consistent set of logical clocks, one per process.

Logical Clocks: Solution

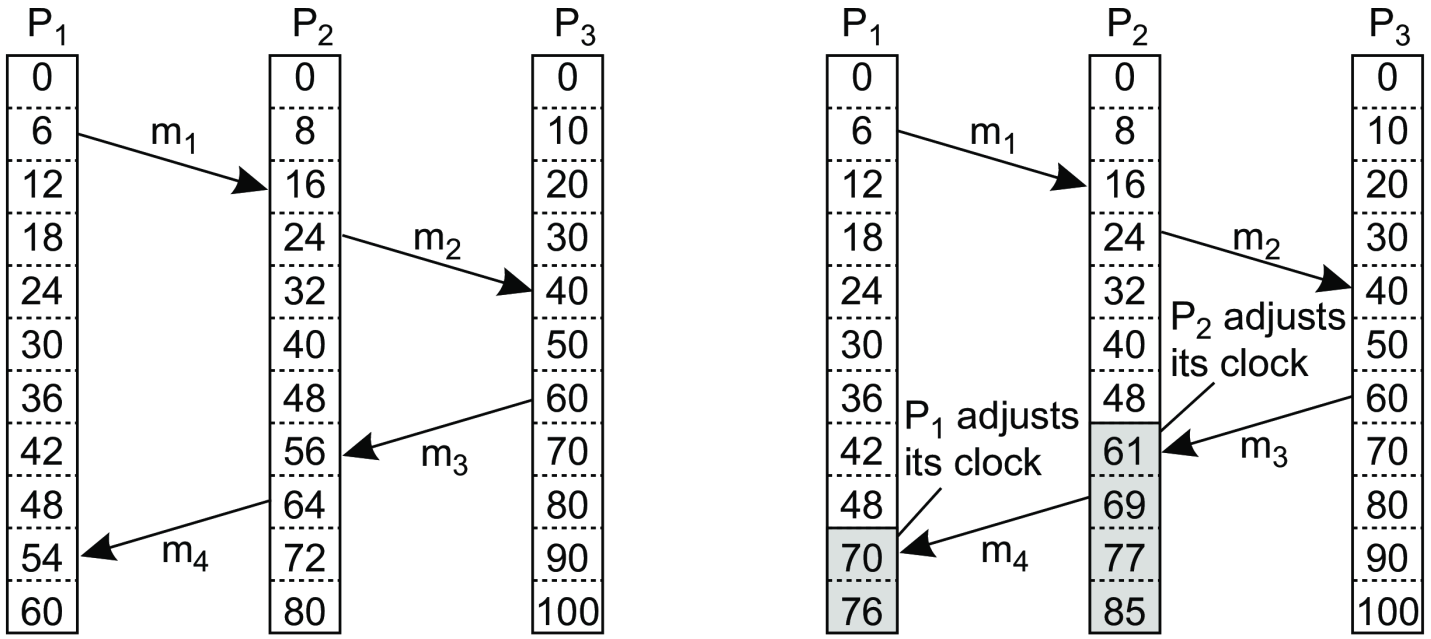
- Each process P_i maintains a local counter C_i and adjusts this counter
  1. For each new event that takes place within P_i , C_i is incremented by 1.
  2. Each time a message m is sent by process P_i , the message receives a timestamp $ts(m) = C_i$.
  3. Whenever a message m is received by a process P_j , P_j adjusts its local counter C_j to $\max\{C_j, ts(m)\}$; then executes step 1 before passing m to the application.

Note 17.2.

- Property P1 is satisfied by (1); Property P2 by (2) and (3).
- It can still occur that two events happen at the same time. Avoid this by breaking ties through process IDs.

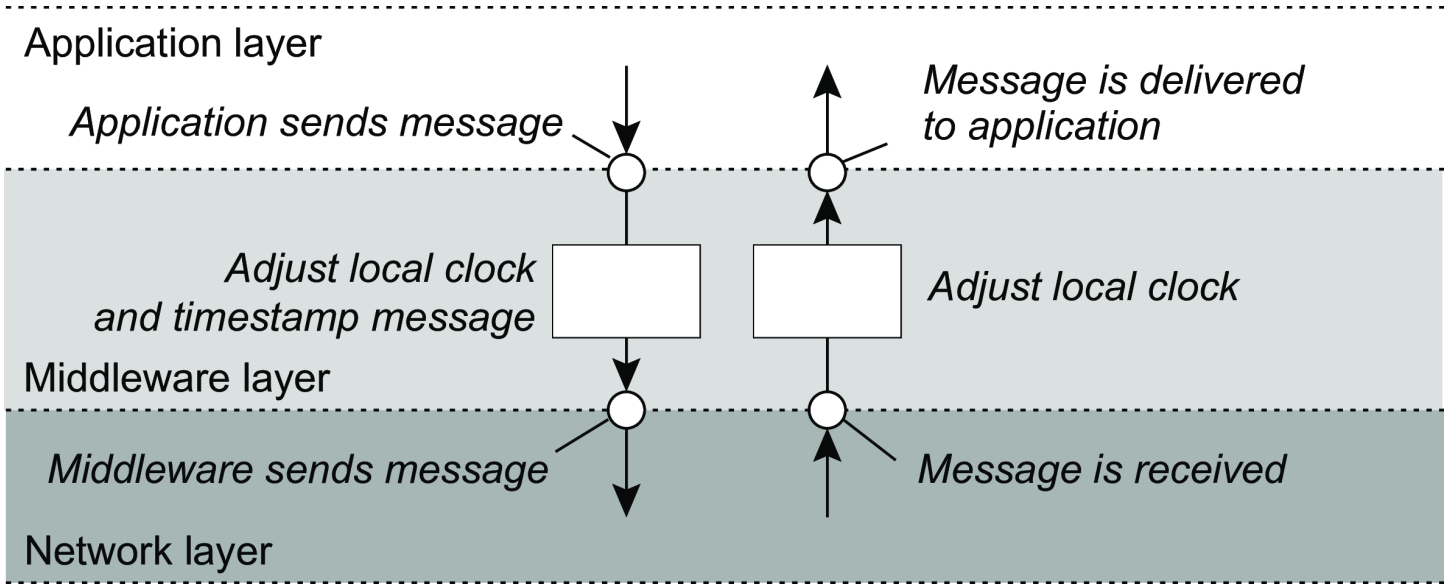
Logical Clocks: Example

- Consider three processes with event counters operating at different rates:



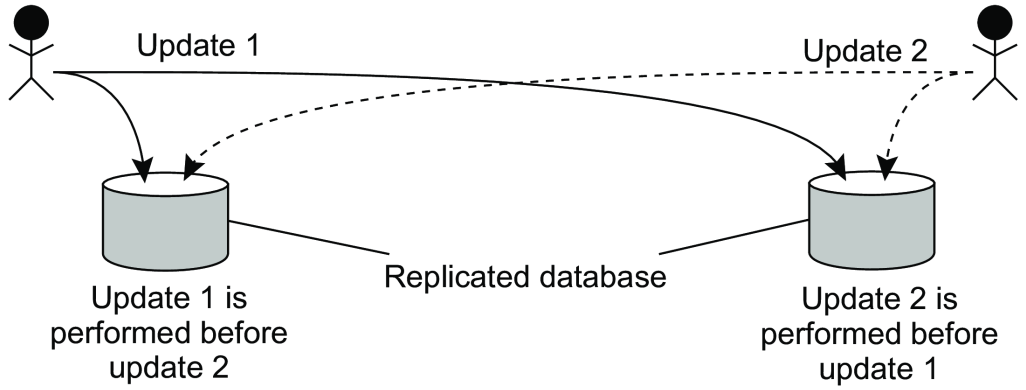
Logical Clocks: Where Implemented

- Adjustments implemented in middleware



Example: Totally Ordered Multicast

- Concurrent updates on a replicated database are seen in the same order everywhere
  - P1 adds $100 to an account (initial value: $1000)
  - P2 increments account by 1%
  - There are two replicas



- In absence of proper synchronization: replica #1 ← $1111, while replica #2 ← $1110.

Example: Totally Ordered Multicast

- **Solution**
  - Process P_i sends timestamped message m_i to all others. The message itself is put in a local queue $queue_i$.
  - Any incoming message at P_j is queued in $queue_j$, according to its timestamp, and acknowledged to every other process.
- P_j passes a message m_i to its application if:
  1. m_i is at the head of $queue_j$
  2. for each process P_k , there is a message m_k in $queue_j$ with a larger timestamp.

Note 17.3.

We are assuming that communication is reliable and FIFO ordered.

Lamport's Clocks For Mutual Exclusion

```
1  class Process:
2    def __init__(self, chanID, procID, procIDSet):
3      self.chan.join(procID)
4      self.procID     = int(procID)
5      self.otherProcs.remove(self.procID)
6      self.queue      = []                    # The request queue
7      self.clock      = 0                     # The current logical clock
8
9    def requestToEnter(self):
0      self.clock = self.clock + 1                         # Increment clock value
1      self.queue.append((self.clock, self.procID, ENTER)) # Append request to q
2      self.cleanupQ()                                     # Sort the queue
3      self.chan.sendTo(self.otherProcs, (self.clock, self.procID, ENTER)) # Send request
4
5    def ackToEnter(self, requester):
6      self.clock = self.clock + 1                         # Increment clock value
7      self.chan.sendTo(requester, (self.clock, self.procID, ACK)) # Permit other
8
9    def release(self):
0      tmp = [r for r in self.queue[1:] if r[2] == ENTER]  # Remove all ACKs
1      self.queue = tmp                                    # and copy to new queue
2      self.clock = self.clock + 1                         # Increment clock value
3      self.chan.sendTo(self.otherProcs, (self.clock, self.procID, RELEASE)) # Release
4
5    def allowedToEnter(self):
6      commProcs = set([req[1] for req in self.queue[1:]]) # See who has sent a message
7      return (self.queue[0][1] == self.procID and len(self.otherProcs) == len(commProcs))
```

Lamport's Clocks For Mutual Exclusion

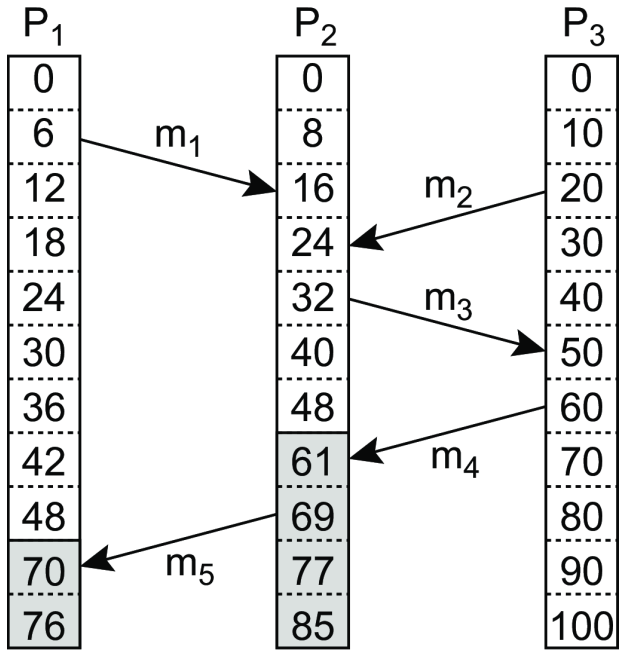
```
1   def receive(self):
2     msg = self.chan.recvFrom(self.otherProcs)[1]      # Pick up any message
3     self.clock = max(self.clock, msg[0])              # Adjust clock value...
4     self.clock = self.clock + 1                       # ...and increment
5     if msg[2] == ENTER:
6       self.queue.append(msg)                          # Append an ENTER request
7       self.ackToEnter(msg[1])                         # and unconditionally allow
8     elif msg[2] == ACK:
9       self.queue.append(msg)                          # Append a received ACK
0     elif msg[2] == RELEASE:
1       del(self.queue[0])                              # Just remove first message
2     self.cleanupQ()                                   # And sort and cleanup
```

Lamport’s Clocks For Mutual Exclusion

- **Analogy with totally ordered multicast**
  - With totally ordered multicast, all processes build identical queues, delivering messages in the same order
  - Mutual exclusion is about agreeing in which order processes are allowed to enter a critical region

Vector Clocks

- Lamport's clocks do not guarantee that if C(a) < C(b) that a causally preceded b.
- Concurrent message transmission using logical clocks
- Event a: m1 is received at T = 16;
- Event b: m2 is sent at T = 20.
- We cannot conclude that a causally precedes b.



Causal Dependency

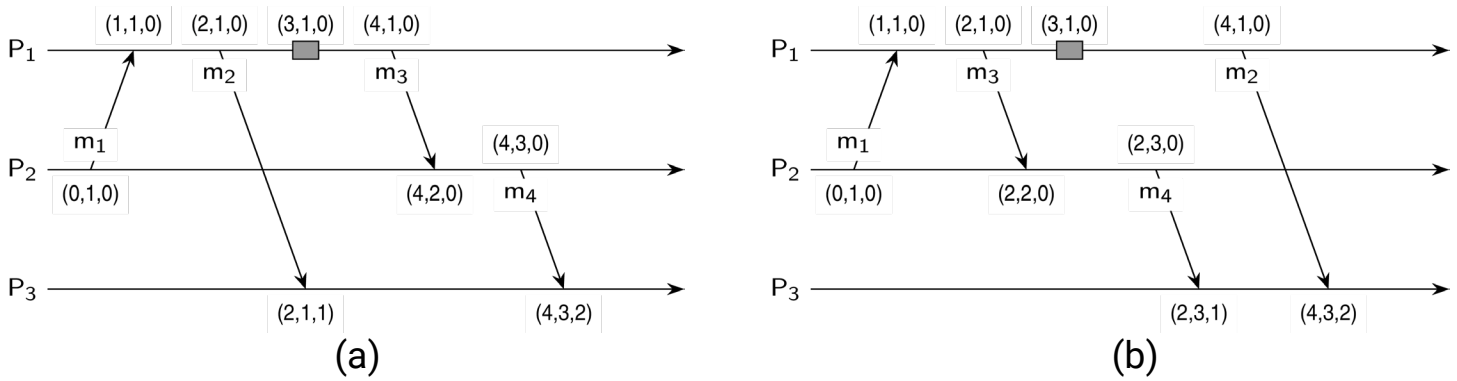
- We say that b may causally depend on a if $ts(a) < ts(b)$, with:
  - for all k , $ts(a)[k] \leq ts(b)[k]$and
  - there exists at least one index k' for which $ts(a)[k'] < ts(b)[k']$
- **Precedence vs. dependency**
  - We say that a causally precedes b.
  - b may causally depend on a, as there may be information from a that is propagated into b.

Capturing Potential Causality

- **Solution: each P_i maintains a vector VC_i**
  - $VC_i[i]$ is the local logical clock at process Pi .
  - If $VC_i[j] = k$ then P_i knows that k events have occurred at P_j .
- **Maintaining vector clocks**
  1. Before executing an event, P_i executes $VC_i[i] \leftarrow VC_i[i] + 1$.
  2. When process P_i sends a message m to P_j , it sets m's (vector) timestamp ts(m) equal to VC_i after having executed step 1.
  3. Upon the receipt of a message m, process P_j sets $VC_j[k] \leftarrow \max\{VC_j[k], ts(m)[k]\}$ for each k , after which it executes step 1 and then delivers the message to the application.

Vector Clocks: Example

- **Capturing potential causality when exchanging messages**



- **Analysis**

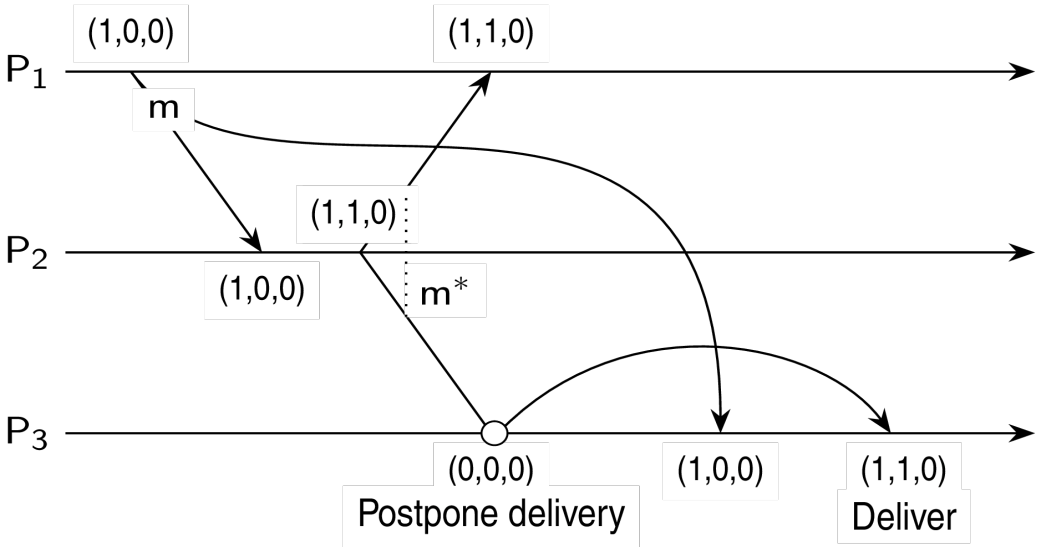
Situation	$ts(m_2)$	$ts(m_4)$	$ts(m_2) < ts(m_4)$	$ts(m_2) > ts(m_4)$	Conclusion
(a)	(2, 1, 0)	(4, 3, 0)	Yes	No	m_2 may causally precede m_4
(b)	(4, 1, 0)	(2, 3, 0)	No	No	m_2 and m_4 may conflict

Causally Ordered Multicasting

- We can now ensure that a message is delivered only **if all causally preceding messages** have already been deivered.
- P_i increments $VC_i[i]$ only when sending a message, and P_j "adjusts" VC_j when receiving a message (i.e., effectively does not change $VC_j[j]$).
- **P_j postpones delivery of m until:**

Causally Ordered Multicasting

- Enforcing causal communication



- Example
- Take VC3 = [0, 2, 2], ts(m) = [1, 3, 0] from P1. What information does P3 have, and what will it do when receiving m (from P1)?

Wrap-Up + What's Next

In the next lecture, we will discuss the Mutual Exclusion, including:

- Centralized Algorithm,
- Distributed Algorithm,
- Token-ring Algorithm,
- Decentralized Algorithm.